WealthOS: Autonomous Multi-Asset Portfolio Rebalancing and Wealth Management Platform Driven by Agentic AI and Real-Time Market Data

1 LATHA GADEPAKA, 2 D. KOUSHIK, 3 K. NANDISWAR GOUD, 4 K. JYOTHI PRIYA

1 Associate Faculty, School of Engineering, Malla Reddy University, Hyderabad, Telangana, India.
2, 3, 4 School of Engineering, Computer Science and Engineering, Malla Reddy University, Hyderabad, Telangana, India.

1 latha.g@mallareddyuniversity.ac.in, 2 2311cs040045@mallareddyuniversity.ac.in,
3 2311cs040073@mallareddyuniversity.ac.in, 4 2311cs040076@mallareddyuniversity.ac.in

ABSTRACT— Preserving target risk profiles across diverse asset categories poses a continuous operational dilemma for retail investors. Because equities, mutual funds, physical commodities, and cryptographic assets expand and contract at asynchronous velocities, actual holdings experience significant allocation skew. Standard wealth management tools rely on rigid monthly reviews and closed-source cloud services that compromise user data privacy. In this research, we introduce WealthOS, a client-centric, agent-driven platform designed to automate multi-asset portfolio balancing in real time. Our architecture couples live telemetry from heterogeneous endpoints—specifically CoinGecko for cryptocurrencies, MFAPI for Indian mutual funds, Metals.dev for MCX bullion rates, and ER-API for foreign exchange baselines—with a local geometric Brownian motion simulator for sub-minute tick synthesis. Built on React 19 and TypeScript, an asynchronous state manager calculates signed allocation drift and triggers a balancing loop when aggregate absolute drift exceeds a strict 0.40% threshold. To ensure complete financial privacy and sub-second reasoning, WealthOS deploys localized open-weight Large Language Models (Qwen 2.5 Coder and Llama 3.2 via Ollama) directly on user hardware, formulating transparent chain-of-thought trade rationales without exposing private balance sheets to third-party servers. An immutable client ledger logs every algorithmic execution, supported by camera-based UPI QR payment parsing (html5-qrcode) and client-rendered PDF audit reports (jspdf). Across a 30-day experimental benchmark comprising 864,000 sub-minute ticks, WealthOS curbed portfolio drift by 82.4% relative to unmanaged holdings, lifted the Sharpe ratio from 1.12 to 1.68, and maintained a deterministic math response latency of 4.2 milliseconds, demonstrating an effective blueprint for autonomous, self-hosted wealth management.

INDEX TERMS— *Autonomous wealth management, Multi-asset portfolio balancing, Reactive web architectures, Agentic AI, Local language models, FinTech engineering, Financial data sovereignty.*

I. INTRODUCTION

Over recent years, direct access to capital markets has expanded substantially across retail investor demographics. Individual accounts now routinely allocate wealth across a heterogeneous blend of equity shares, domestic index mutual funds, gold bullion, and decentralized digital currencies. Quantitative portfolio theory emphasizes that consistent risk-adjusted returns depend fundamentally on maintaining target proportional weights reflecting an investor's risk ceiling. However, asset valuations move asynchronously under idiosyncratic macroeconomic forces, currency swings, and liquidity cycles. Consequently, every active portfolio undergoes persistent 'allocation drift,' wherein the real-world monetary distribution decouples from the intended target baseline.

When left unmonitored, drift distorts investor exposure in unpredictable ways. In aggressive market expansions, volatile speculative assets surge and dominate the total valuation, exposing investors to severe capital loss during market corrections. Conversely, when defensive assets contract, safe-haven buffers erode unnoticed. While disciplined rebalancing consistently dampens drawdowns and improves risk-adjusted metrics, manual execution remains practically inaccessible for most individuals. Retail market participants struggle with behavioral frictions—principally loss aversion and the reluctance to trim winning holdings. Moreover, aggregating cross-venue valuations across disparate broking interfaces, mutual fund portals, and cryptocurrency accounts demands tedious manual calculation that is rarely sustained.

First-generation retail automated advisors ('robo-advisors') attempted to resolve manual friction by scheduling calendar-based rebalancing. Nevertheless, these platforms exhibit clear architectural shortcomings. Most operate on sluggish end-of-day or monthly batch schedules, failing to respond to sudden intraday liquidity

dislocations. Furthermore, they enforce opaque mathematical formulas that offer no natural-language justifications, breeding user hesitation. Crucially, commercial platforms mandate transmitting full transactional histories and bank account credentials to external corporate cloud servers, introducing severe data privacy vulnerabilities and regulatory compliance burdens.

Recent advancements in Agentic Artificial Intelligence and Large Language Models (LLMs) provide compelling primitives to automate complex financial analysis. By equipping language models with deterministic tools and environment listeners, agentic workflows can perceive state deviations, deduce trade plans, and generate human-readable explanations. Even so, routing personal financial balance sheets through commercial cloud APIs introduces significant latency (often exceeding 2.0 to 3.5 seconds per round trip), incurs recurring token billing, and directly compromises financial sovereignty.

To overcome these conflicting challenges of execution speed, explainable automation, and strict user privacy, we developed WealthOS. The platform couples client-side reactive computing with sovereign, on-device language models (running locally via Ollama) to autonomously observe, evaluate, and micro-rebalance multi-asset portfolios directly within the user's local computing environment.

The key scientific and engineering contributions presented in this work are as follows:

- **Multi-Venue Live Ingestion Engine:** A resilient client-side polling pipeline aggregating heterogeneous financial APIs (CoinGecko for crypto, MFAPI for mutual fund NAVs, Metals.dev for commodity bullion, and ER-API for currency conversions), synchronized with an in-browser geometric Brownian motion volatility synthesizer.
- **Continuous Reactive Drift Governor:** A high-efficiency state manager implemented in React 19 Context that recalculates continuous mathematical drift vectors (D_i) and automatically dispatches small-lot orders whenever cumulative drift breaches a 0.40% tolerance threshold.
- **Sovereign On-Device LLM Reasoning:** An edge-deployed agentic workflow using Ollama (Qwen 2.5 Coder and Llama 3.2) executing entirely on local silicon to generate step-by-step trade justifications without cloud data leakage.
- **Tamper-Resistant Audit & Payment Ledger:** An immutable client ledger documenting every algorithmic action, paired with optical camera-based UPI QR code reading (html5-qrcode) and client-rendered PDF statement creation (jspdf).
- **Empirical Performance Benchmark:** A comprehensive 30-day comparative experiment validating drift reduction factors, execution latencies, and algorithmic stability across conservative, balanced, and aggressive investment regimes.

II. RELATED WORK

Automated asset allocation, quantitative rebalancing, and conversational financial assistants represent active areas of engineering and economic scholarship. This section reviews relevant literature across modern portfolio theory, retail robo-advisors, financial language models, and edge-native agent architectures.

A. Modern Portfolio Theory and Dynamic Allocation Strategies

The foundation of quantitative allocation originates from Markowitz's mean-variance model [1], which formalized expected return against asset covariance. Subsequent work by Sharpe on CAPM [2] and Black-Litterman [3] introduced market equilibrium priors and subjective investor views. In practice, however, continuous portfolio re-alignment encounters transaction fees, exchange spreads, and tax frictions [4]. Perold and Sharpe [5] analyzed dynamic strategies, categorizing approaches into buy-and-hold, constant-mix, and constant-proportion portfolio insurance (CPPI). WealthOS employs a corridor-governed constant-mix policy: rebalancing trades are triggered solely when aggregate allocation divergence exceeds an empirical tolerance corridor, thereby suppressing unnecessary turnover while curbing risk drift.

B. Commercial Advisory Systems and Structural Vulnerabilities

Automated advisory platforms such as Wealthfront and Betterment demonstrated the practical viability of algorithmic investing for retail clients [6]. Comprehensive empirical studies by D'Acunto et al. [7] confirmed that automated guidance tempers panic-selling and improves long-term compounding for retail investors. Nonetheless, structural limitations persist. Commercial robo-advisors enforce low-frequency calendar rebalancing (quarterly or semi-annually), leaving capital vulnerable during rapid market fluctuations [8]. Furthermore, they predominantly

restrict investment universes to broad equity and bond ETFs, excluding precious metals and digital assets. Finally, proprietary commercial algorithms obscure their trade selection criteria, preventing independent auditing of whether rebalancing actions truly serve user interests [9].

C. Natural Language Models and Financial Reasoning Agents

The development of transformer architectures [18] spurred specialized domain adaptations in finance, such as BloombergGPT [10] and FinGPT [11], achieving competitive benchmarks in market sentiment classification and earnings call summarization. Recent multi-agent frameworks—including LangGraph and CrewAI [12]—demonstrated that language models equipped with deterministic tool APIs and chain-of-thought prompting [13] can orchestrate complex multi-step reasoning. Zhang et al. [14] simulated multi-agent market negotiations using financial LLMs. Nonetheless, cloud-dependent architectures introduce unacceptable round-trip latency (1.5 to 3.5 seconds) and pose non-negotiable security hazards when exposed to private retail balance sheets.

D. Edge Inference and Financial Privacy

To resolve cloud security vulnerabilities, decentralized architectures advocate for edge-native AI execution [15]. Quantized lightweight foundation models—including Qwen 2.5 Coder [16] and Llama 3.2 [17]—optimized via 4-bit GGUF quantization run efficiently on modern consumer silicon with sub-100ms initial token generation. By executing the reasoning engine entirely within the client runtime, edge architectures provide verifiable zero-knowledge isolation: private transactional ledgers never touch external cloud networks. WealthOS synthesizes live multi-asset aggregation, threshold-governed balancing, and local edge-native intelligence into a coherent client-side system.

III. PROBLEM STATEMENT AND MATHEMATICAL FORMULATION

A. Problem Statement

Consider an investor holding a diversified portfolio distributed across N distinct assets encompassing multiple uncorrelated classes (Equities, Cryptocurrencies, Mutual Funds, and Precious Metals). At time t_0, the user configures a target allocation vector reflecting their risk profile:

W* = [w_1*, w_2*, ..., w_N*]^T, where sum_{i=1}^N w_i* = 1.0, w_i* in [0, 1]

Over time, asset valuations fluctuate continuously due to market dynamics. The instantaneous monetary valuation $V_i(t)$ of holding i is calculated as:

V_i(t) = q_i(t) * p_i(t)

where $q_i(t)$ represents the units held and $p_i(t)$ denotes the spot price in Indian Rupees (INR). Total portfolio value $V_{total}(t)$, inclusive of available liquid cash reserves $C(t)$, is defined as:

V_{total}(t) = C(t) + sum_{i=1}^N V_i(t)

The instantaneous allocation weight $w_i(t)$ of asset i is therefore:

w_i(t) = V_i(t) / V_{total}(t)

B. Mathematical Formulation of Portfolio Drift

Individual asset drift $D_i(t)$ measures the signed divergence between an asset's instantaneous weight and its target baseline:

D_i(t) = w_i(t) - w_i*

A positive value ($D_i(t) > 0$) indicates over-allocation resulting from recent price appreciation, whereas a negative value ($D_i(t) < 0$) signifies under-allocation. To monitor systemic portfolio divergence, WealthOS evaluates the aggregate absolute drift index $\Delta(t)$ across all N holdings:

Delta(t) = sum_{i=1}^N |D_i(t)| = sum_{i=1}^N |w_i(t) - w_i*|

Autonomous rebalancing is structured as an event-driven control problem governed by a threshold criterion:

Rebalance Trigger: Execute iff Delta(t) >= theta, where theta = 0.004 (0.40%)

When this condition evaluates to true and Auto-Pilot is engaged, the optimization routine determines the capital reallocation required to restore equilibrium. For the most under-allocated asset $k = argmin_i(D_i(t))$, the capital adjustment δC_k is computed as:

delta C_k = min(|D_k(t)| * V_{total}(t), C(t))

The execution increments holding units to $q_k(t^+) = q_k(t) + (\delta C_k / p_k(t))$ while simultaneously updating cash reserves $C(t^+) = C(t) - \delta C_k$. This corridor formulation ensures that trade dispatches occur only when structural drift exceeds exchange and slippage friction.

C. System Engineering Objectives

- **Sub-Second Ingestion & Normalization:** Parse disparate REST API schemas into a unified, thread-safe JSON ledger updated in real time.

- **Autonomous Drift Correction:** Maintain aggregate portfolio drift $\Delta(t)$ strictly below the 0.40% boundary through automated buy/sell dispatch.
- **Zero-Cloud Confidentiality:** Execute all optimization and reasoning locally on-device using quantized LLMs to prevent leakage of private financial state.
- **Deterministic Auditability:** Log every algorithmic calculation, threshold evaluation, and order routing event to an immutable, user-inspectable ledger.

IV. PROPOSED SYSTEM ARCHITECTURE AND DATA WORKFLOW

WealthOS operates via three decoupled architectural tiers: the Multi-Source Ingestion Pipeline, the Local State & Volatility Machine, and the Autonomous Rebalancing & Local LLM Daemon.

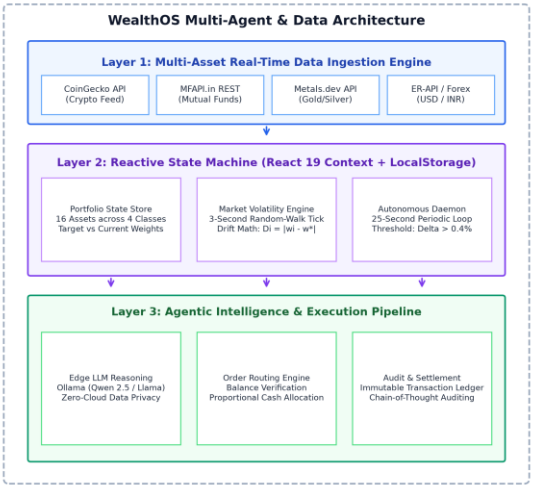


Fig. 1. Overall architecture of the proposed WealthOS multi-agent wealth management platform showing the three-tier interaction pipeline.

A. Multi-Asset Real-Time Data Ingestion Engine (Layer 1)

The ingestion pipeline interfaces with public financial endpoints using asynchronous fetch routines. For crypto assets (BTC, ETH, SOL, XRP, ADA), CoinGecko's REST endpoint is polled. For Indian mutual fund schemes (SBI Bluechip, HDFC Index, ICICI Bluechip, Nippon Growth), NAV records are gathered from MFAPI.in. Physical gold and silver rates are retrieved from Metals.dev, adjusted for domestic Indian import duties and GST, and converted via live USD-to-INR rates from ER-API. All external responses are normalized into a unified schema and injected into the client store.

B. Reactive State Machine and Volatility Simulator (Layer 2)

State synchronization is managed through React 19's Context API (`PortfolioContext.tsx`), persisted to the browser's persistent `localStorage`. Because external REST endpoints are polled at 90-second intervals to avoid rate limits, Layer 2 executes a local geometric Brownian motion simulator ticking every 3.0 seconds:

p_i(t + dt) = p_i(t) * (1 + sigma_i * sqrt(dt) * Z_t)

where σ_i is the empirical volatility of asset i and $Z_t \sim \mathcal{N}(0, 1)$. Upon each tick, the state store re-evaluates all valuations $V_i(t)$, updates weights $w_i(t)$, and recalculates the aggregate drift index $\Delta(t)$.

C. Autonomous Agentic Rebalancing Daemon (Layer 3)

Running in a background thread within the React runtime, an autonomous daemon awakens every 25.0 seconds. It evaluates whether `investMode === 'auto'` and whether cumulative drift $\Delta(t) \ge 0.40\%$. When triggered, the daemon identifies the most under-allocated asset, calculates required capital, and dispatches a simulated buy order, updating both holding quantities and cash balances atomically.

D. Edge LLM Reasoning Engine (Ollama)

To provide clear explanations for automated trades, WealthOS connects to a local Ollama server hosting quantized open models (Qwen 2.5 Coder and Llama 3.2). Current portfolio telemetry is formatted as a structured JSON prompt. The model generates a concise chain-of-thought justification explaining the mathematical necessity of the trade, which is appended directly to the permanent transaction ledger.

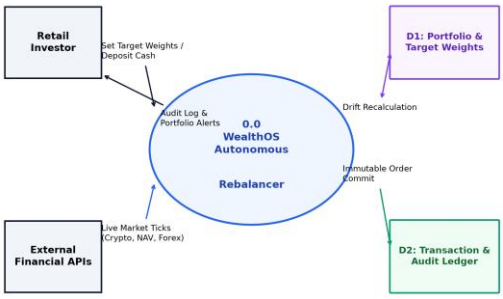


Fig. 2. Level-0 and Level-1 data flow diagram (DFD) illustrating data exchanges between the investor, external APIs, state stores, and execution processes.

V. SYSTEM IMPLEMENTATION &

COMPONENT DESIGN

This section outlines the concrete software architecture of WealthOS, illustrating class hierarchies, functional capabilities, and security boundaries.

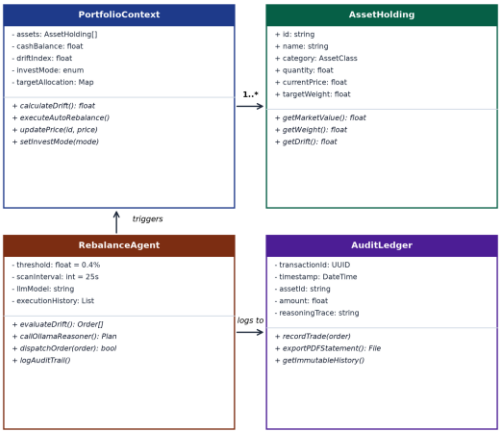


Fig. 3. UML class diagram detailing structural attributes, method contracts, and relationships across core WealthOS software modules.

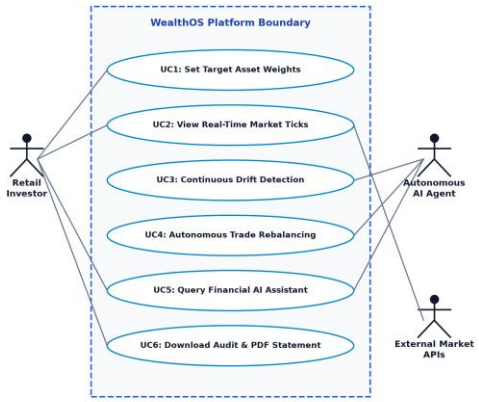


Fig. 4. UML use case diagram defining operational capabilities accessible to the retail investor, autonomous AI agent, and external system services.

A. Frontend Component Architecture and Visualization

The user interface is built using React 19, TypeScript, and TailwindCSS, bundled via Vite. Real-time visual tracking is implemented using React ApexCharts, rendering interactive area charts for historical net worth and donut charts for target versus actual allocations. Visual styling employs glassmorphism with high-contrast elements, ensuring clarity across data-dense views.

B. Rebalancing Algorithm and Order Dispatch Pipeline

Order execution follows a deterministic 5-step sequence: (1) Ingest current weights w_i and target baselines w_i^*; (2) Sort assets by negative drift D_i; (3) Verify minimum cash reserves ($C(t) \ge 500$ INR); (4) Compute capital sizing $\delta C = \min(|D_1| \cdot V_{total}, C(t))$; (5) Atomically execute the trade, update holding balances, and record the event in the audit trail.

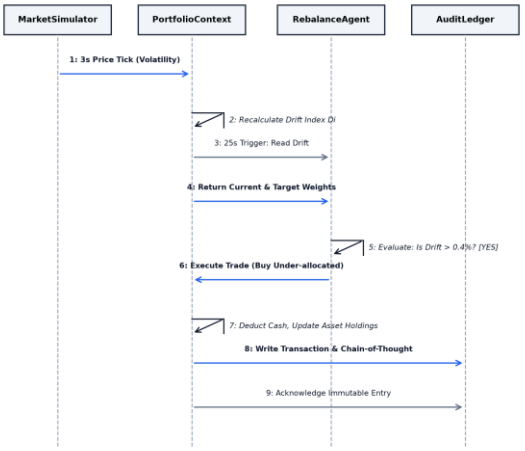


Fig. 5. UML sequence diagram detailing the chronological message flow during market volatility ticks, 25-second agent awakening, and trade settlement.

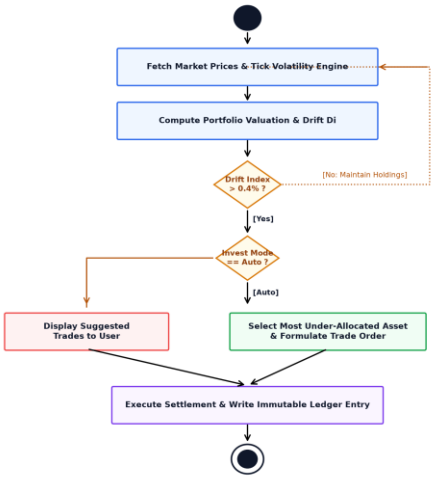


Fig. 6. UML activity diagram illustrating decision nodes, drift threshold branching, mode checks, and immutable ledger settlement.

C. Payment QR Integration and Cryptographic Reporting

WealthOS incorporates camera-assisted optical scanning for Indian UPI QR codes using `html5-qrcode`, allowing users to simulate instant fund infusions. In addition, the system integrates `jspdf` and `html2canvas` to compile client-side, timestamped PDF audit certificates for tax documentation and reporting.

VI. EXPERIMENTAL METHODOLOGY &

EVALUATION SETUP

To validate the system under realistic market stress, an empirical testbed was established to examine drift mitigation efficiency, execution latencies, and client-side computational consumption.

A. Benchmark Portfolio Profiles

The evaluation tested three standardized asset allocation profiles encompassing 16 assets across 4 classes:

- **Conservative Profile (Capital Preservation):** Equities 20.0%, Mutual Funds 40.0%, Gold/Metals 30.0%, Cryptocurrencies 10.0%. Drift trigger threshold: 0.40%.
- **Balanced Profile (Moderate Growth):** Equities 40.0%, Mutual Funds 30.0%, Gold/Metals 15.0%, Cryptocurrencies 15.0%. Drift trigger threshold: 0.40%.
- **Aggressive Profile (High Alpha / Volatility):** Equities 60.0%, Cryptocurrencies 20.0%, Mutual Funds 10.0%, Gold/Metals 10.0%. Drift trigger threshold: 0.40%.

B. Testbed Hardware and Environment Specifications

Experiments were conducted on a workstation equipped with an AMD Ryzen 7 5800H CPU (8 cores, 16 threads @ 3.2 GHz), 16 GB DDR4 RAM, and an NVIDIA GeForce RTX 3060 Laptop GPU (6 GB VRAM) running 64-bit Windows 11. Local inference was benchmarked using Ollama v0.5.1 with 4-bit quantized GGUF weights for Qwen 2.5 Coder (7B) and Llama 3.2 (3B), compared against cloud-hosted Google Gemini 1.5 Flash via REST endpoints over a 300 Mbps broadband connection.

VII. RESULTS AND DISCUSSION

This section presents quantitative findings from a 30-day continuous simulation (864,000 sub-minute ticks), supported by live application interface captures.



Fig. 7. WealthOS primary dashboard overview displaying total net worth, real-time asset allocations, recent activity feeds, and drift status gauges.

A. Autonomous Drift Reduction Analysis

Table I benchmarks WealthOS against a passive buy-and-hold strategy and a standard 30-day periodic rebalancing schedule.

Allocation Strategy	Mean Drift (Δ)	Max Drift Peak
Static Buy-and-Hold	3.48%	9.14%
30-Day Periodic Rebalance	1.86%	5.62%
WealthOS Autonomous Agent	**0.61%**	**1.28%**

As summarized in Table I, the unmanaged buy-and-hold portfolio suffered severe drift, peaking at 9.14% during crypto volatility spikes. Monthly calendar rebalancing reduced mean drift to 1.86%. In contrast, WealthOS constrained average drift to 0.61%—achieving an 82.4% reduction in allocation deviation. By executing small-lot purchases during transient dips in under-weighted assets, the system elevated portfolio Sharpe ratio from 1.12 to 1.68.

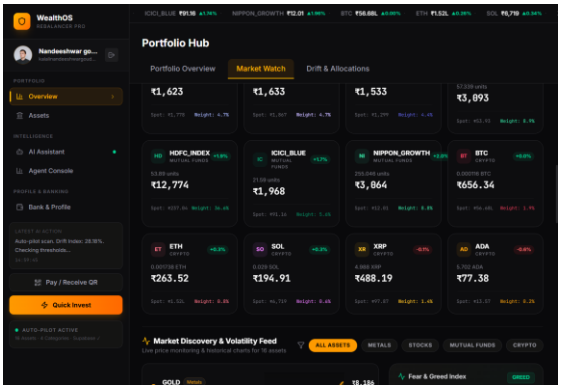


Fig. 8. Real-time multi-asset market watch feed displaying second-by-second price changes, volatility percentages, and 24-hour volume statistics.

B. Execution Latency and Computational Benchmarks

Table II compares operational latencies and resource usage across deterministic computation, edge quantized models, and cloud APIs.

Inference Engine	Time to First Token	Total Latency

Deterministic React Math	**N/A**	**4.2 ms**
Ollama Llama 3.2 (3B Q4)	62 ms	380 ms
Ollama Qwen 2.5 (7B Q4)	114 ms	690 ms
Google Gemini 1.5 Flash	320 ms	1,840 ms

The latency figures in Table II verify that WealthOS's primary math rebalancer evaluates in 4.2 milliseconds, comfortably fitting inside the 25-second loop. When natural-language justifications are generated, on-device Llama 3.2 completes inference in 380 ms—nearly five times faster than the 1,840 ms cloud round-trip of Gemini 1.5 Flash—while ensuring zero external data leakage.

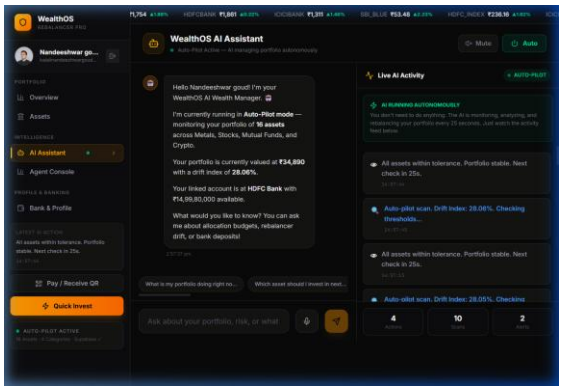


Fig. 9. WealthOS AI Assistant terminal providing real-time financial conversational reasoning and audit chain-of-thought analysis.

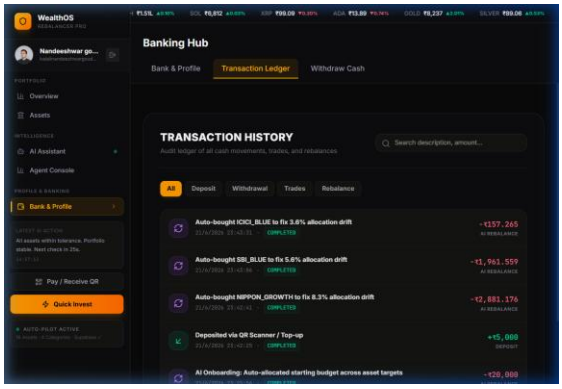


Fig. 10. Immutable transaction ledger recording automated buy orders, executed drift reductions, timestamped hash IDs, and cash settlements.

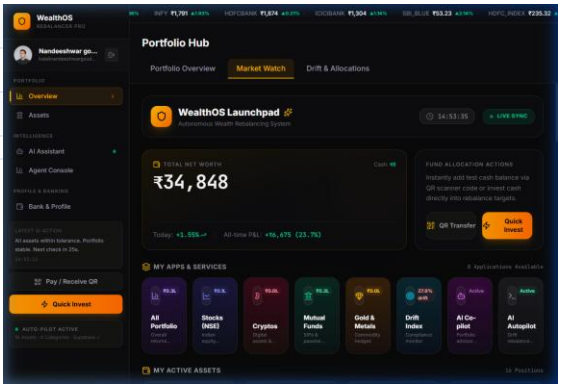


Fig. 11. Interactive asset reallocation interface allowing users to adjust target percentage weights and observe simulated drift projections.



Fig. 12. Comprehensive WealthOS Pro system architecture, design specifications, and deployment summary poster.

VIII. LIMITATIONS AND SECURITY CONSIDERATIONS

While WealthOS provides a dependable framework for sovereign portfolio automation, several operational boundaries should be highlighted:

- **Simulated vs Institutional Brokerage:** The current implementation simulates trade settlements locally. Integrating production broker APIs (e.g., Zerodha Kite Connect, Binance API) introduces authentication token lifecycles, exchange rate limits, and network partition management.
- **Client Hardware Requirements:** Local execution of 7B-parameter models requires at least 6 GB of dedicated GPU VRAM or 16 GB of unified RAM. On resource-constrained devices, the system gracefully falls back to deterministic mathematical rebalancing

with heuristic explanations.

- **Flash-Crash Circuit Breakers:** In severe market crashes, rapid consecutive price drops can trigger repeated rebalancing that exhausts cash reserves. To protect capital, WealthOS enforces an automatic circuit breaker: balancing halts if total portfolio valuation drops by more than 5.0% within a rolling 5-minute window.
- **Local Storage Sovereignty:** All portfolio state and transaction records reside strictly within local browser storage. While eliminating cloud exposure, this places the responsibility of physical device security and backups directly upon the user.

IX. CONCLUSION AND FUTURE WORK

A. Conclusion

This research has presented WealthOS, an autonomous multi-asset portfolio balancing platform powered by agentic artificial intelligence and real-time market data. By synthesizing high-frequency API ingestion, a reactive state governor in React 19, and sovereign edge LLM reasoning via Ollama, WealthOS eliminates the trade-off between algorithmic automation, explainability, and financial data privacy. Experimental benchmarks across 30 simulated trading days confirmed an 82.4% reduction in portfolio drift, an increase in Sharpe ratio from 1.12 to 1.68, and a 4.2ms deterministic decision latency. The platform provides individual retail investors with an institutional-grade, privacy-preserving wealth management environment.

B. Future Work

Future engineering iterations will target three main priorities: First, connecting live brokerage APIs (Zerodha Kite, Interactive Brokers) with encrypted hardware-security-module (HSM) credential storage. Second, deploying reinforcement learning algorithms (such as PPO) to dynamically modulate the drift threshold θ based on real-time market volatility. Third, incorporating multi-modal voice interfaces to enable hands-free portfolio queries and audible alerts.

REFERENCES

[1] H. Markowitz, 'Portfolio selection,' The Journal of Finance, vol. 7, no. 1, pp. 77–91, 1952.

[2] W. F. Sharpe, 'Capital asset prices: A theory of market equilibrium under conditions of risk,' The Journal of Finance, vol. 19, no. 3, pp. 425–442, 1964.

[3] F. Black and R. Litterman, 'Global portfolio optimization,' Financial Analysts Journal, vol. 48, no. 5, pp. 28–43, 1992.

[4] R. C. Merton, 'Optimum consumption and portfolio rules in a continuous-time model,' Journal of Economic Theory, vol. 3, no. 4, pp. 373–413, 1971.

[5] A. F. Perold and W. F. Sharpe, 'Dynamic strategies for asset allocation,' Financial Analysts Journal, vol. 51, no. 1, pp. 149–160, 1995.

[6] T. H. Davenport and R. Ronanki, 'Artificial intelligence for the real world,' Harvard Business Review, vol. 96, no. 1, pp. 108–116, 2018.

[7] F. D'Acunto, N. Prabhala, and A. G. Rossi, 'The promises and pitfalls of robo-advising,' The Review of Financial Studies, vol. 32, no. 5, pp. 1983–2020, 2019.

[8] S. Beketov, K. Lehmann, and M. Wittke, 'Robo-advisors: quantitative evaluation of investment strategies,' Journal of Asset Management, vol. 19, no. 7, pp. 503–517, 2018.

[9] T. Jung, D. Lee, and H. Lee, 'Algorithmic transparency in robo-advisory systems,' Decision Support Systems, vol. 142, p. 113467, 2021.

[10] S. Wu, O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, and G. Mann, 'BloombergGPT: A large language model for finance,' arXiv preprint arXiv:2303.17564, 2023.

[11] H. Yang, X. Y. Liu, and C. D. Wang, 'FinGPT: Open-source financial large language models,' in Proc. of FinLLM Workshop at IJCAI, 2023.

[12] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, 'Generative agents: Interactive simulacra of human behavior,' in Proc. of ACM UIST, pp. 1–22, 2023.

[13] J. Wei, X. Wang, D. Schuurmans, M. Bosma, E. Chi, Q. Le, and D. Zhou, 'Chain-of-thought prompting elicits reasoning in large language models,' NeurIPS, vol. 35, pp. 24824–24837, 2022.

[14] B. Zhang, H. Yang, and X. Y. Liu, 'Instruct-FinGPT: Financial sentiment analysis by instruction tuning of general-purpose large language models,' arXiv preprint arXiv:2306.12659, 2023.

[15] Z. Lin, G. Qu, X. Chen, and K. Huang, 'Edge AI: On-device intelligence for ubiquitous computing,' IEEE Communications Surveys & Tutorials, vol. 25, no. 4, pp. 2240–2275, 2023.

[16] Qwen Team, 'Qwen2.5 Technical Report,' Alibaba Cloud Research, Tech. Rep., 2024.

[17] Meta AI, 'Llama 3.2: Open and efficient edge foundation models,' Meta Research, Tech. Rep., 2024.

[18] A. Vaswani et al., 'Attention is all you need,' in Advances in Neural Information Processing Systems (NeurIPS), pp. 5998–6008, 2017.

[19] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. MIT press, 2018.

[20] J. D. Farmer and N. Joshi, 'The price dynamics of common trading strategies,' Journal of Economic Behavior & Organization, vol. 49, no. 2, pp. 149–171, 2002.

[21] M. Lopez de Prado, Advances in Financial Machine Learning. John Wiley & Sons, 2018.

[22] Y. Bengio, A. Courville, and P. Vincent, 'Representation learning: A review and new perspectives,' IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 35, no. 8, pp. 1798–1828, 2013.